

Kolhapur Institute of Technology's
College of Engineering , Kolhapur (Empowered Autonomous)

“Café Management System”

Submitted in partial fulfillment of the requirement of the degree of

Mini Project-I

(UCBIL0371)

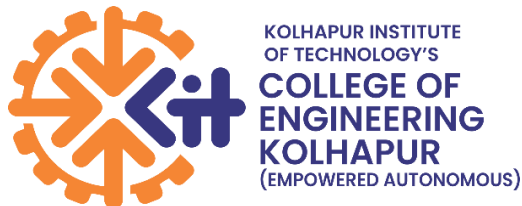
by

- | | | |
|-----|------------------------------|-------------------|
| 36. | <i>Prajwal Suresh Jadhav</i> | <i>2425000517</i> |
| 57. | <i>Sarthak Vinayak Patil</i> | <i>2425000940</i> |
| 47. | <i>Soham Umesh Hasolkar</i> | <i>2425000691</i> |

Under the Supervision of

J.S.Pujari

Designation, Department of CSBS



Department Computer Science and Business Systems

KITCOE, Kolhapur

October, 2025



**Kolhapur Institute of Technology's
College of Engineering Kolhapur (Empowered Autonomous)
(Department of Computer Science and Business Systems)**

CERTIFICATE

*This is to certify that following students of Second Year Computer Science and Business Systems has submitted Mini Project-I report entitled “**Café Management System**” in partial fulfillment for the course code “**UCBIL0371**”, in the academic year 2025-26. It has been found to be satisfactory and hereby approved for the submission.*

1 Prajwal Jadhav	2425000517
2 Sarthak Patil	2425000940
3 Soham Hasolkar	2425000691

J.S.Pujari
Guide

Dr. Manjunath Hudagi
Head, Dept. of CSBS

INDEX

S.NO	TOPIC	PAGE NO
1.	INTRODUCTION	5
2.	LITERATURE REVIEW	6
3.	STATEMENT OF PROBLEM	7
4.	OBJECTIVES OF THE STUDY 4.1 Core Functional Objectives (What the System Must Do) 4.2 Technical and Educational Objectives (How the System is Built) 4.3 Non-Functional Objectives (Quality Goals)	8-10
5.	SOFTWARE REQUIREMENT SPECIFICATION 5.1 Functional Requirements 5.2 Non-Functional Requirements 5.3 Hardware Requirements	11-14
6.	DETAILED DESIGN 6.1 Data flow Diagram 6.2 Use Case Diagram 6.3 Activity Diagram 6.4 Architecture Diagram	15-17
7.	IMPLEMENTATION 7.1 Admin Module 7.2 Customer Module 7.3 Data Management and File Handling 7.4 System Architecture	18-19

8.	RESULTS AND DISCUSSION 8.1 Admin Module Testing 8.2 Customer Module Testing 8.3 Functional Performance 8.4 Advantages Observed 8.5 Limitations and Improvement Areas 8.6 Discussion	20-25
9.	CONCLUSION & FUTURE SCOPE 9.1 Conclusion 9.2 Future Scope 9.3 Final Thoughts	26-29
10.	REFERENCES	30

1. INTRODUCTION

Running a café is about more than just serving great coffee. It's about managing daily operations smoothly from taking customer orders and maintaining the menu to generating bills and keeping track of sales. For many small cafés, this often involves handwritten notes, registers, or spreadsheets, which can be time-consuming and error-prone.

To address this, we've developed a **Café Management System**, a simple yet efficient console-based application built using the **C programming language** and **core concepts of Data Structures and Algorithms (DSA)**. This project brings essential café operations into a structured digital format that's easy to use, reliable, and fast.

The system allows staff to **add and update menu items, place or cancel orders, generate bills instantly, and store customer feedback**. At its core, the application uses **linked lists, queues, and stacks** to handle dynamic menu management, order processing, and billing efficiently. This use of DSA ensures that operations remain quick and organized, even as the number of customers or menu items grows.

The idea behind this project is simple: to **apply classroom learning to real-world situations**. By simulating a real café environment, we created a system that not only demonstrates programming and data structure concepts but also solves practical business problems.

With this system, café staff can focus more on customer service and less on managing records manually. It reduces workload, minimizes human error, and saves time proving how thoughtful programming can make everyday work easier and more efficient.

2.LITERATURE REVIEW

Technology has become a big part of the food and hospitality industry. Today, many cafés and restaurants rely on automated systems to take orders, update menus, track sales, and serve customers more efficiently. But while larger businesses can afford such advanced systems, many small cafés still depend on old methods handwritten bills, registers, or simple spreadsheets. These manual processes take time, are easy to mess up, and can slow things down, especially when the café is busy.

Most of the café management software available in the market comes with great features, but they often require expensive subscriptions, powerful systems, or a constant internet connection. This makes them difficult for smaller cafés to use. What many of these businesses really need is a **simple, affordable, and easy-to-use system** that works on any basic computer without much setup.

Our **Café Management System** is designed to fill that gap. It takes the best ideas from professional systems but keeps them simple. Built using **C programming** and **core Data Structures and Algorithms (DSA)**, it focuses on essential features like **adding menu items, placing and canceling orders, generating bills, managing feedback and expenses**. The entire system works offline and doesn't require any expensive software or hardware.

We also wanted to show how **DSA can solve real-world problems**, not just academic exercises. By using **linked lists, queues, and stacks**, the system keeps everything organized and fast, even when multiple customers are being served at once.

In short, this project blends learning with practicality. It's built for small cafés that want something efficient yet simple, while also proving how smart programming can make everyday business smoother and more manageable.

3. STATEMENT OF PROBLEM

Many small cafés struggle with organizing their daily tasks like order processing, billing , menu management . Without an efficient system, errors happen and customers have to wait longer. This project focuses on building an easy-to-use café management system that reduces mistakes and speeds up service .

4. OBJECTIVES OF THE STUDY

The core objectives of the **Café Management System** mini-project are to develop a simple, robust, and efficient console-based application that applies Data Structures and Algorithms (DSA) concepts to streamline the daily operations of a small to medium-sized café.

The project aims to solve the problem of manual inefficiency, common in small cafés, which often leads to disorganized order processing, slow service, and billing errors.

4.1 Core Functional Objectives (What the System Must Do)

▪ Simplified Menu Management

A key objective is to provide the **administrator (staff with privileges)** with an easy, lightweight tool to manage the café menu dynamically.

- **Adding Items:** Allow the admin to rapidly **add new items** (name and price) to the menu, utilizing a linked list structure for dynamic memory allocation.
- **Updating Items:** Enable the admin to **modify existing item details**, such as price and quantity, by searching the menu.
- **Removing Items:** Provide a straightforward function to **delete any item** from the menu by ID or name.
- **Canceling Items:** Allow the admin to **temporarily set a menu item as unavailable** without removing it entirely. **Displaying Menu:** Offer a neatly formatted display of all available menu items to both administrators and customers.

▪ Efficient Order Processing

The system must digitize the order workflow to ensure speed and accuracy, reducing customer wait times.

- **Taking Orders:** Allow staff to accept multiple items and quantities from a customer and immediately **add the order to a queue** for kitchen processing, thereby enforcing a **First-In, First-Out (FIFO)** service model.

- **Processing Orders:** Implement a function to process and send the next customer order from the queue to the kitchen, maintaining an organized flow.
- **Accurate Billing and Sales Tracking**

A primary goal is to eliminate human errors in financial transactions and provide administrative oversight.

- **Generating Bills:** Utilize a stack to temporarily store and calculate the final bill, including the total amount, taxes, and any discounts, before printing the detailed output.
- **Viewing Bills:** Allow the customer and/or administrator to **view the details of a current or historical order bill.**
- **Expense Tracking:** Provide the admin with a function to check and track the café's operational expenses. User Feedback and Customer Management

Provide a dedicated module for customers to submit valuable feedback and for administrators to manage customer relations.

- **Submitting Feedback:** Provide a dedicated module for customers to **submit valuable feedback** about their experience.
- **Reviewing Feedback:** Enable the admin to **view and manage all customer feedback** submitted through the system.
- **Customer Details Management:** Provide the admin with the functionality to view relevant customer details for loyalty, communication, and service purposes.

4.2 Technical and Educational Objectives (How the System is Built)

- **Practical DSA Implementation:** To demonstrate the application of core Data Structures and Algorithms in a real-world scenario, specifically:
 - Using **Linked Lists** for dynamic and flexible menu management.
 - Using **Queues** for structured, FIFO-based order handling.
 - Using **Stacks** for efficient bill generation and calculation.

- **C Programming Language Proficiency:** To gain practical experience in building a complete system using the C programming language, focusing on file handling and memory management.
- **Console-Based Lightweight Design:** To maintain a simple and accessible system by using a console interface, which is suitable for basic hardware and emphasizes the back-end logic.

4.3 Non-Functional Objectives (Quality Goals)

- **Performance:** The system must be fast, aiming to process menu updates, order placements, and bill generation within 1–2 seconds to ensure smooth café operations even during busy hours.
- **Security:** Ensure basic role-based security where the Customer Module does not have privileges to alter menu items or prices, reserving that functionality for the Admin Module.
- **Portability:** The application must be compatible with any system capable of running a C compiler (Windows, Linux, macOS).

In essence, the project is a practical exercise to build an accessible, efficient, and error-reducing management tool that transitions a manual café process into a structured, digital system powered by fundamental programming concepts

5. SOFTWARE REQUIREMENT SPECIFICATION

5.1 Functional Requirements

Module 1: Admin Module

▪ Functions :

1. **addItem()**

- Description: Allows admin to add new item to the menu (name, price) using dynamic memory (linked list node creation).
- Input: Item name, price, and other details.
- Expected Output: Confirmation message indicating the item was successfully added.

2. **removeMenuItem()**

- Description: Removes an item from persistent storage **Menu.txt** by item ID. writes remaining items to a temporary file, then replaces the original file.
- Input: Item ID .
- Expected Output: Success/failure message & **Menu.txt** updated.

3. **displayMenuItem()**

- Description: Displays the current menu for admin review.
- Input: No specific input required.
- Expected Output: A neatly formatted list of all menu items with IDs, names, prices.

4. **viewfeedback()**

- Description: Prints stored feedback from **Feedback.txt** in formatted block.
- Input: No specific input required.
- Expected Output: Tabular feedback display.

5. **checkexpense()**

- Description: Aggregate expenses by date into list & Prints daily totals and Prints stored customer information & order recorded from **customerdetails.txt**.

- Input: No specific input required.
- Expected Output: Daily expenses report printed to console.

6.customerdetails()

- Description: Prints stored customer information and order records from customerdetails.txt to the console.
- Input: No specific input required.
- Expected Output: Display of customer names, mobile numbers, order details, and associated

Module 2: Customer Module

▪ Functions :

1. viewMenu()

- Description: Display the menu for customer from **Menu.txt**.
- Input: No specific input required.
- Expected Output: Display of the current menu with item IDs, names, prices, and availability.

2. placeOrder()

- Description: Allows staff to accept multiple items and quantities from a customer's order. The order is added to a queue for kitchen processing (FIFO).
- Input: Item IDs and quantities from customer.
- Expected Output: Confirmation message including date, ID ,Price.

3. cancelMenu()

- Description: Removes specific item from order list by item ID.
- Input: Order item ID.
- Expected Output: Success/failure message & order list updated.

4. viewbill()

- Description: prints a bill to the console, computes the total & appends order details along with customer info to **customerdetails.txt** for record keeping.
- Input: No specific input required.
- Expected Output: Printed bill with itemized costs & total.

5.feedback()

- Description: Takes customer's name, mobile number & text feedback and appends it to **Feedback.txt**. Prints a thank-you message.
- Input: Name, Mobile number , feedback.
- Expected Output: **Feedback.txt** updated and confirmation printed.

5.2 Non-Functional Requirements

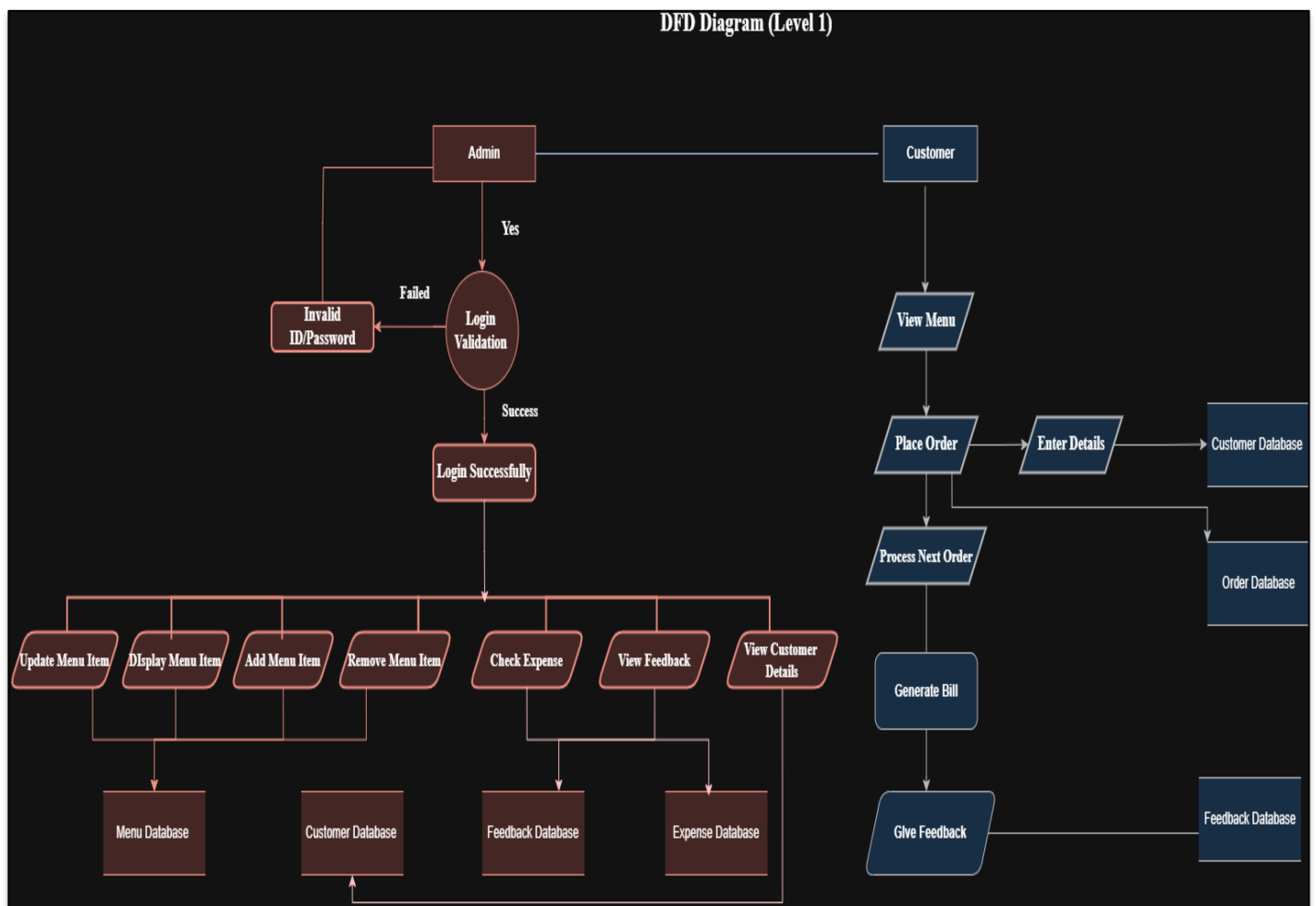
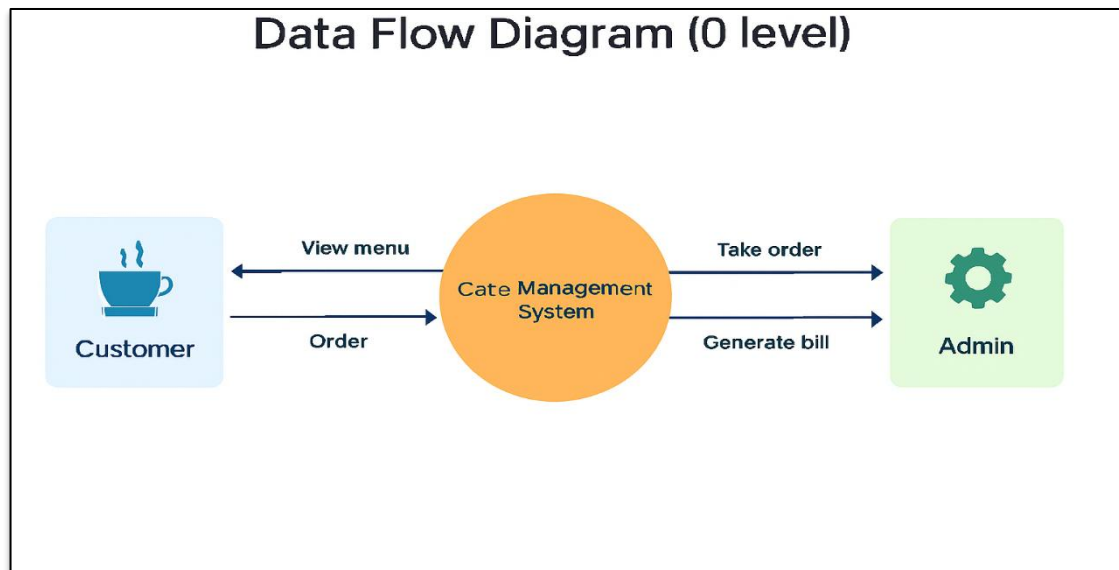
1. **Performance**: The system should process menu updates, order placements, and bill generation within 1–2 seconds for smooth café operations.
2. **Security**: Customer module should not have privileges to alter menu or prices.
3. **Portability**: The application should be compatible with any system running a C compiler (Windows, Linux, macOS).

5.3 Hardware Requirements

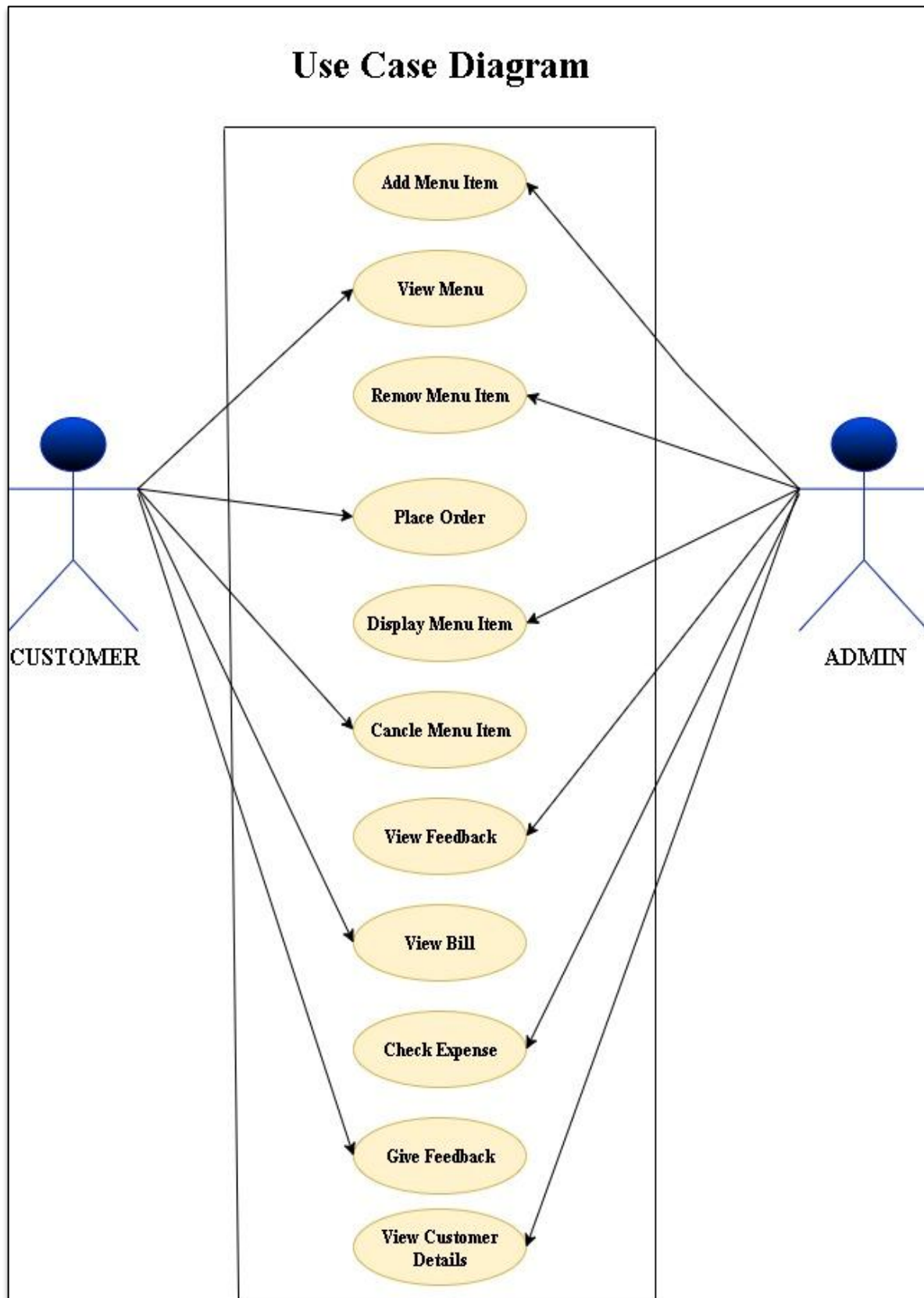
- **Processor**: Minimum Intel/AMD dual-core (1 GHz or higher)
- **RAM**: Minimum 2 GB
- **Storage**: At least 100 MB free disk space
- **Input Devices**: Keyboard (mandatory), Mouse (optional)
- **Other**: System capable of running a C compiler (Dev c++ , VS code)

6.DETAILED DESIGN

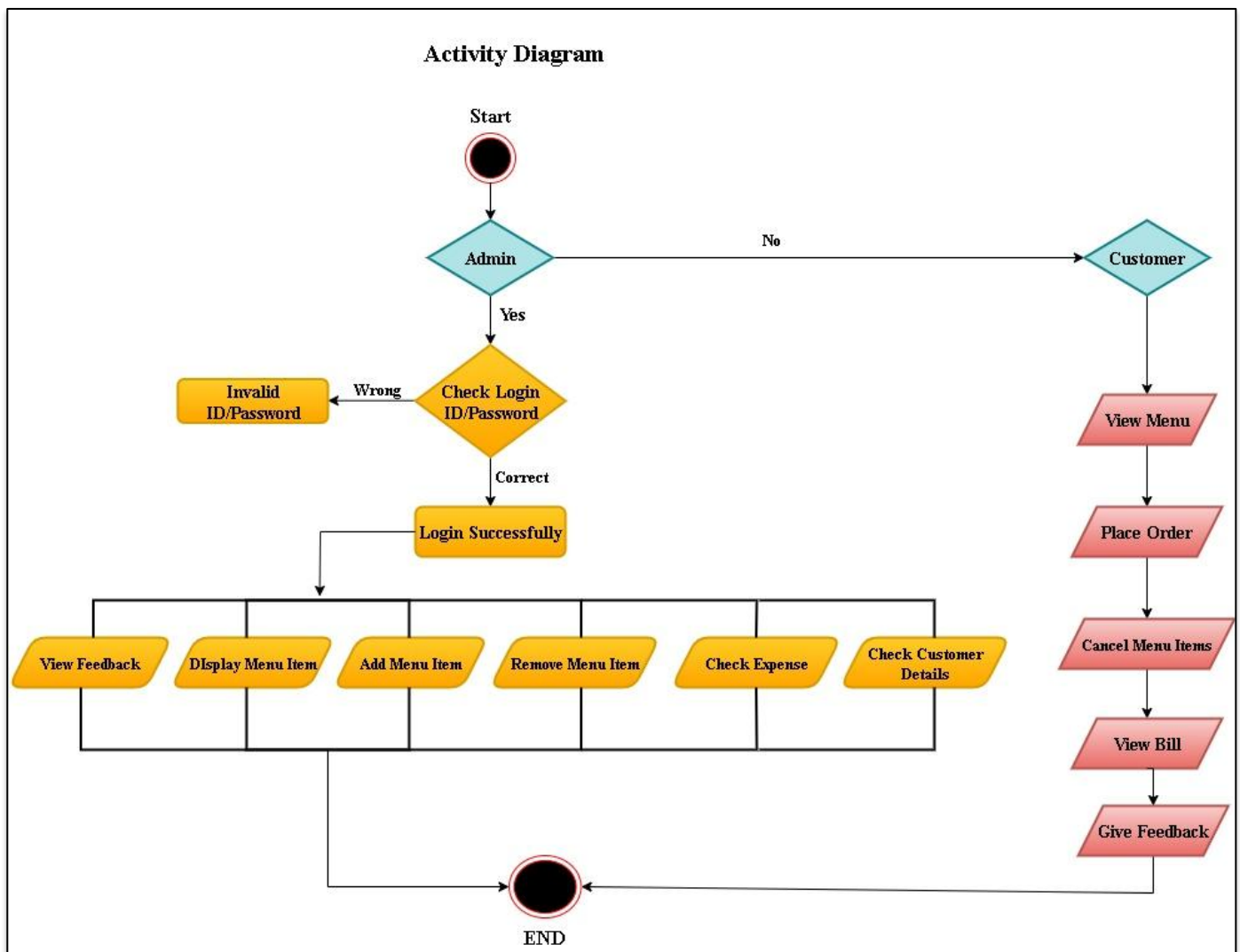
6.1 Data flow Diagram



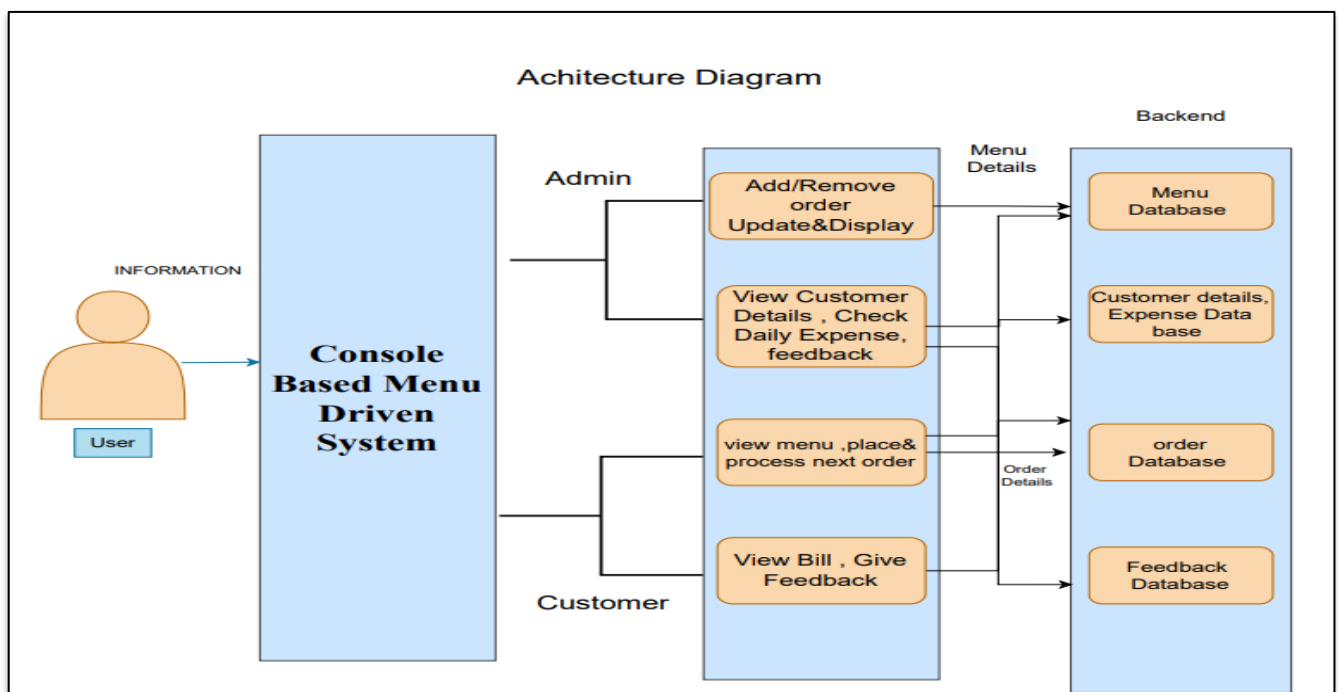
6.2 Use Case Diagram



6.3 Activity Diagram



6.4 Architecture Diagram



7.IMPLEMENTATION

The Café Management System is implemented in the **C programming language**, utilizing fundamental **Data Structures and Algorithms (DSA)** to make the system efficient, organized, and easy to maintain. The implementation is divided into two main modules: **Admin** and **Customer**, each handling distinct functions of the café's operations.

7.1 Admin Module

The Admin Module is **secured with a login prompt** that requires checking a valid ID and password. Upon successful login, the administrator gains access to the full management suite. This module provides options to **Add Menu Item, Remove Menu Item, Display Menu Item, View Feedback, Check Expense, and Check Customer Details**. The admin can dynamically manage the café's menu without editing any files manually. These menu items are stored and managed through **linked lists and file handling**, ensuring data persistence even after the program is closed.

7.2 Customer Module

The Customer Module is the primary interface for users who do not log in as an administrator. It allows customers to easily **View Menu, Place Order, Cancel Menu Items** (from their current order), **View Bill**, and **Give Feedback**. Orders are managed using **linked lists** to allow flexible and real-time order processing. Once an order is placed, it is stored in memory, and the total bill is generated automatically, making the process quick and accurate.

7.3 Data Management and File Handling

File handling plays a key role in the system's persistence and portability. The Menu.txt file stores all menu items, and the Feedback.txt file records customer feedback for the admin to review later. This design removes the need for any external database, making the system lightweight and portable. Furthermore, files would likely be used to store login credentials for the Admin and potentially for recording sales/expense data ('Check Expense').

7.4 System Architecture

Control structures like loops, conditional statements, and switch-case logic are used extensively for robust menu-driven navigation and workflow logic, mirroring the flow shown in the Activity Diagram. Each operation is implemented as a **separate function**, making the code modular and easy to maintain. The system runs smoothly on basic hardware without any external dependencies, making it ideal for small cafés looking for a simple, fast, and effective digital solution to manage their daily operations.

8. RESULTS AND DISCUSSION

The **Café Management System** was successfully developed and executed using the **C programming language** along with fundamental **Data Structures and Algorithms (DSA)**. The project achieved its primary goal to provide a simple, efficient, and cost-effective solution for managing café operations such as menu handling, order processing, bill generation, and feedback collection.

The system was tested on multiple scenarios to ensure it functions correctly and provides accurate results for both the **Admin** and **Customer** modules. The outputs demonstrated reliable performance, user-friendly navigation, and quick processing of tasks. The program is entirely console-based, yet it manages to provide a structured workflow similar to a basic point-of-sale (POS) system.

8.1 Admin Module Testing

The Admin module was tested with different inputs and scenarios to verify that menu operations work as expected.

```
Enter 1 for Admin
Enter 2 for Customer
Enter 3 for Exit
1
Enter the password (int only) : 12345
-----
Enter your choice:
1 - Add menu item
2 - Remove menu item
3 - Display menu items
4 - Display feedback
5 - Check expense
6 - View Customer Details
7 - Exit
Choice: |
```

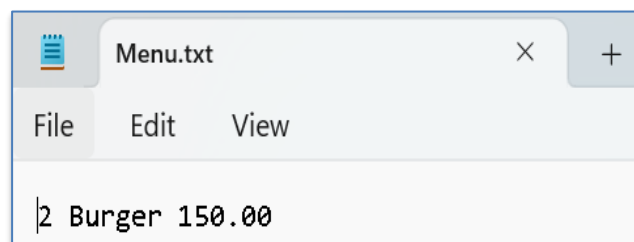
- **Adding Menu Items:** The admin can enter new items along with their prices. Once added, the items appear in the menu list instantly and are saved in the Menu.txt file. This ensures that even if the program is closed, the updated menu is retained.

```
Choice: 1
Enter the name of the item: pizza
Enter the price of the item: 199
Item added successfully with ID 1.
Choice: 1
Enter the name of the item: Burger
Enter the price of the item: 150
Item added successfully with ID 2.
```

Menu.txt		
File	Edit	View
1	pizza	199.00
2	Burger	150.00

- **Removing Menu Items:** The system allows the admin to remove items using their unique ID. If the ID does not exist, an error message is displayed.

```
Choice: 2
Enter id to remove item : 1
Item removed successfully!
```



- **Displaying Menu Items:** All menu items are displayed in a structured tabular format. This feature is crucial for keeping the menu updated for customers in real time.

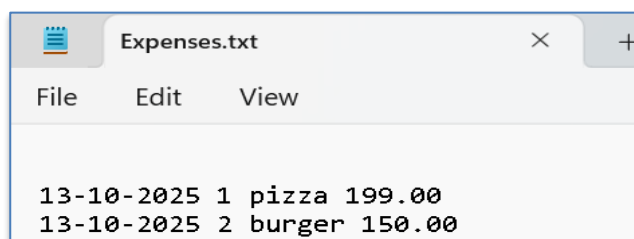
```
Choice: 3

      MENU CARD
-----
ID      Item Name      Price
-----
2       Burger         150.00 Rs
-----
```

- **Check Expense:** Testing confirmed that the admin can execute the 'Check Expense' function to view and track operational costs or sales data, ensuring the necessary files or data structures are correctly accessed and presented.

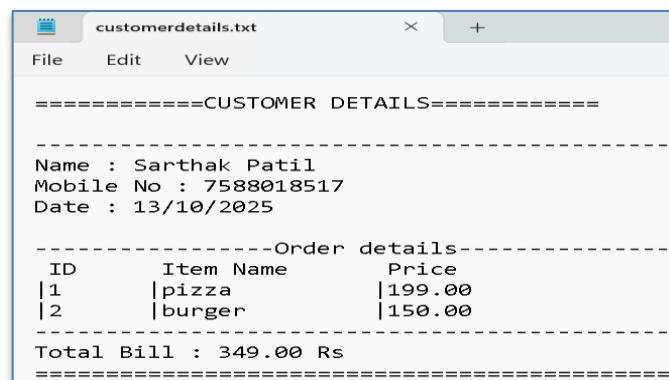
```
Choice: 5

--- Daily Expense Report ---
Date: 13-10-2025 | Total Expense: 349.00 Rs
```



- **Check Customer Details:** The 'Check Customer Details' function was tested to ensure the admin can successfully retrieve and view customer information, typically for loyalty, order history, or contact purposes.

```
Choice: 6
=====CUSTOMER DETAILS=====
-----
Name : Sarthak Patil
Mobile No : 7588018517
Date : 13/10/2025
-----Order details-----
ID      Item Name      Price
|1      |pizza         |199.00
|2      |burger         |150.00
-----
Total Bill : 349.00 Rs
=====
```



- **Feedback Viewing:** All customer feedback is stored and retrieved from Feedback.txt. This allows the admin to read suggestions or complaints later, which can help improve service quality.

Choice: 4

|| FEEDBACK ||

Name	Mobile No	Feedback
Sarthak Patil	7588018517	Excellent Products!!!

Feedback.txt

File Edit View H1

FEEDBACK

Name	Mobile No	Feedback
Sarthak Patil	7588018517	Excellent Products!!!

8.2 Customer Module Testing

The Customer module was designed to be simple, clear, and quick to use. It was tested thoroughly for usability and accuracy.

```

Enter 1 for Admin
Enter 2 for Customer
Enter 3 for Exit
2

Enter your choice from below
1 - View menu
2 - Place order
3 - remove Order
4 - View bill
5 - Feedback
6 - Exit
Choice: |

```

- **Viewing Menu:** Customers can view the available menu items directly from the file. This ensures they always see the latest version updated by the admin.

|| MENU CARD ||

ID	Item Name	Price
1	pizza	199.00 Rs
2	burger	150.00 Rs
3	coffee	90.00 Rs
4	cake	250.00 Rs

- **Placing Orders:** Orders can be placed by entering the item ID. Multiple items can be added in a single session, and they are stored using a linked list.

```

Choice: 2
Enter the id to order menu :
1
Id : 1   Name : pizza   Price : 199.00
Item added succesfully !

```

- **Canceling Orders:** Customers can cancel specific items from their order. This helps in modifying the bill in real time without restarting the process.

```
Choice: 3
Enter the item id to cancel your order :
1
Your order removed succesfully !
```

- **Bill Generation:** After placing the order, the system generates the total bill with item details and prices. This output is instant and accurate.

```
Choice: 4

=====
              BILL
=====
ID      Item Name      Price
=====
1       pizza          199.00
2       burger          150.00
=====
Total Bill: 349.00 Rs
```

- **Feedback Submission:** Customers can submit their name, mobile number, and feedback. This data is stored in a text file, ensuring it remains available for the admin.

```
Choice: 5
Enter your name: Sarthak Patil
Enter your mobile number: 7588018517
Please enter your valuable feedback:
Excellent Products!!!

Thank You For Your Valuable Response!!!...
```

8.3 Functional Performance

The performance of the system was evaluated based on **speed**, **accuracy**, and **ease of use**. All basic operations menu updates, order processing, and bill generation are completed in **under 2 seconds** on a basic machine with no internet connectivity. The use of **linked lists** ensures dynamic memory allocation and smooth handling of variable menu sizes and order lists.

The **file handling mechanism** works efficiently to store and retrieve data between sessions without requiring a database. This makes the system lightweight and suitable for small cafés and college canteens.

8.4 Advantages Observed

- **Quick and Reliable:** All functions work in real time, making order placement and billing fast.
- **Simple Interface:** Console-based interaction ensures minimal training required for staff.
- **Data Persistence:** Menu and feedback are stored using text files, so the data is not lost when the program closes.
- **Low Cost:** No internet connection, external database, or special hardware required.
- **Modular Code:** Functions are well-separated, making future modifications easier.

8.5 Limitations and Improvement Areas

While the system works effectively, a few limitations were observed during testing:

- No **graphical user interface (GUI)**, which may be more appealing for commercial use.
- Limited **error handling** for some edge cases, like invalid input formats.
- Currently supports **single-user** operation at a time; multi-user functionality could enhance real-world use.
- Feedback is stored in a basic text file advanced analytics or dashboards are not available yet.

8.6 Discussion

The implementation of the Café Management System highlights how fundamental programming concepts can solve real-world problems. The use of **linked lists** makes menu and order handling flexible, while **file handling** ensures data storage without a database. By keeping the structure simple and the operations clear, the project strikes a good balance between functionality and ease of use.

This project also demonstrates that **even small-scale digital tools can bring efficiency to everyday business operations**. In a real café environment, it can reduce the dependency on manual processes, minimize errors, and make customer service faster.

Moreover, the modular design makes it easy to enhance the system in the future for example, adding a GUI, integrating payment gateways, or connecting to a database like MySQL for larger-scale use.

9. CONCLUSION & FUTURE SCOPE

9.1 Conclusion

The **Café Management System** was designed and developed as a practical and lightweight solution to simplify everyday café operations. The primary aim of this mini project was to demonstrate how fundamental programming concepts and data structures can be applied to solve **real-world business problems** effectively.

Throughout the development process, key concepts of **C programming** including functions, loops, conditionals, file handling, and modular programming were combined with **data structures** such as linked lists, queues, and stacks to create a structured and efficient system. This approach allowed us to build an application that performs smoothly even on low-specification hardware, without the need for external software or database systems.

The system consists of two main modules: the **Admin module** and the **Customer module**.

- The **Admin module** enables secure menu management, feedback review, and system control through password-protected access.
- The **Customer module** provides easy access to menu viewing, order placement, bill generation, and feedback submission.

This modular structure ensures clarity, maintainability, and ease of use. By dividing responsibilities between admin and customer functions, the project closely mirrors the actual operations of a small café or canteen.

One of the most significant achievements of this project is its **simplicity and reliability**. Since the system relies on text files for storing menu items and feedback, there is no dependency on complex databases or expensive software solutions. This makes it ideal for **small businesses, local cafés, college canteens**, and similar setups where affordability and ease of use are key.

The testing phase proved that the system can handle operations like adding or removing menu items, placing multiple orders, canceling

orders, and generating accurate bills in real time. It also demonstrated stable performance, with minimal processing time and consistent output.

From a learning perspective, this project also helped the team strengthen its understanding of **modular programming, data handling, and real-world application of DSA**. It highlights how even basic programming tools can bring real value when applied with a clear problem-solving mindset.

9.2 Future Scope

Although the current version of the Café Management System meets its basic objectives, there is a lot of potential to **expand and enhance** the project. With a few improvements, it can evolve from a console-based application into a more robust and professional café management platform.

Some of the future enhancements that can be implemented include:

1. **Graphical User Interface (GUI):**

Adding a user-friendly GUI using libraries like GTK or integrating with other frameworks would make the system more attractive and easier to navigate for both admin and customers. A clear visual interface could also reduce training time for staff.

2. **Database Integration:**

Replacing text files with a structured database such as MySQL or SQLite would allow better data management, faster retrieval, and more secure storage. This would also make it possible to handle a larger menu and customer base efficiently.

3. **Inventory Management System:**

Integrating inventory tracking would allow the admin to monitor stock levels, get automatic alerts for low inventory, and manage suppliers. This would make the system more complete and useful for actual business operations.

4. Multi user and network access:

Currently, the system operates on a single machine. Adding support for multiple terminals over a local network or cloud integration would enable multiple staff members to use the system simultaneously.

5. Billing Enhancements and Payment Integration:

Future versions could support GST calculation, discounts, and multiple payment methods such as UPI, cards, or digital wallets. This would make it suitable for real commercial use.

6. Order Tracking and Analytics:

By collecting and analyzing order data, the system could provide insights into best-selling items, peak hours, and customer preferences. This data could help café owners make better business decisions.

7. Security and Authentication:

Adding role-based access and encrypted login credentials would make the system more secure. This is especially useful in larger setups where multiple employees have access.

8. Mobile and Web Application:

In the long run, the project could be converted into a mobile or web application, enabling customers to place orders remotely and the admin to manage operations from anywhere.

9.3 Final Thoughts

The Café Management System proves that **big problems can be solved with simple, well-thought-out solutions**. By combining basic programming knowledge with logical system design, we built a working model that improves efficiency, reduces manual errors, and supports better service management.

The future scope of this project is promising. With further development, this system can grow into a full-fledged application capable of handling modern café operations. Whether it's a small college canteen or a local coffee shop, such a system can make daily management smoother, faster, and more organized.

This project is not just a demonstration of coding skills but also a **real example of applying computer science concepts** to improve real-world business processes. It reflects how software solutions can bring structure, accuracy, and speed to traditional industries even with minimal resources.

10. REFERENCES

1. Kernighan, B. W., & Ritchie, D. M. *The C Programming Language*. Prentice Hall.
– Core reference for C language structure and syntax used in the project.
2. Lafore, R. *Data Structures and Algorithms in C*. Sams Publishing.
– Provided guidance for implementing linked lists, queues, and stacks.
3. Balagurusamy, E. *Programming in ANSI C*. Tata McGraw-Hill.
– Referenced for file handling, control structures, and modular programming.
4. GeeksforGeeks–www.geeksforgeeks.org
– Used for quick revision of DSA concepts like linked lists, stacks, and queues.
5. TutorialsPoint–www.tutorialspoint.com
– Helped with file handling, loops, and basic C functions.
6. Programiz–www.programiz.com/c-programming
– Referred for syntax examples and program structure.
7. StackOverflow–www.stackoverflow.com
– Used for solving practical implementation errors.
8. Rao, P., & Nair, K. (2020). “Application of Data Structures in Real-Time Management Systems.” *IJCS&E*.
– Supported the approach of applying DSA to practical systems.
9. Sharma, A., & Patel, D. (2021). “Automation of Small Businesses Using Lightweight Software Solutions.” *JETC*.
– Inspired the lightweight architecture of the system.
10. KITCOE Academic Notes and Lab Manuals.
– Provided conceptual clarity and project structure guidelines.